

ICPC Bootcamp Day 0 Summary

Based on `day0_slides.pptx`

Session Summary

Day 0 introduces the bootcamp, the ICPC contest model, the weekly plan, and the habits students need before they begin solving practice contests. The deck defines ICPC as the International Collegiate Programming Contest: teams of three share one computer, work for a fixed contest window, solve a broad set of algorithmic problems, and submit code to an automatic judge with hidden tests. The main message is that ICPC performance is not only about knowing algorithms. It depends on teamwork, practice, careful reading, fast implementation, disciplined testing, and strategy under time pressure.

The bootcamp logistics are also established. Students will use Kattis for programming practice and eClass for submitting their solutions after practice contests. The daily format combines discussion of previous contest problems, instruction, lunch, and a longer practice contest block. This structure makes review part of the learning loop: students solve, compare approaches, identify mistakes, and then connect those mistakes to the next topic.

The roadmap slide connects each day to the *Competitive Programming 4* textbook. Day 1 introduces algorithm analysis and core data structures. Day 2 covers complete search, recursion, bitmask subset iteration, binary search, and ternary search. Day 3 moves to greedy algorithms and dynamic programming. Day 4 covers graph traversal, flood fill, minimum spanning trees, Kruskal's algorithm, and Dijkstra's shortest path algorithm. Day 5 introduces selected number theory and computational geometry topics. This roadmap positions CP4 as the organizing reference for the week, not as a replacement for solving problems.

The deck also highlights a practical ICPC strategy. At the beginning of a contest, team members should divide the problem set, skim different statements, classify each problem as easy, medium, hard, or unknown, and identify likely problem types such as dynamic programming, MST, or binary search. Easy problems can be attempted quickly, but longer implementations should usually be planned on paper before taking the keyboard. The one-computer format requires explicit communication signals such as "quick test" for a short interruption and "ready to code" for a longer keyboard handoff.

The final part of the deck previews foundational Day 1 material: Big-O analysis, arrays, linked structures, stacks, queues, deques, priority queues, and hash tables. These slides emphasize the contest reason for each concept. Complexity analysis helps teams reject approaches that will time out or exceed memory. Arrays and lists support direct indexed storage and iteration. Stacks solve nested or LIFO problems such as bracket matching. Queues and deques support FIFO or two-ended access patterns. Priority queues retrieve the most important item efficiently, and hash tables support fast membership tests, frequency counts, and key-value mapping.

Key Takeaways

- ICPC is a team programming contest where strategy and communication matter as much as individual coding speed.

- Contest preparation should combine textbook reading, implementation practice, and team practice.
- A good contest start divides reading across teammates, ranks problems, and avoids wasting the shared computer.
- Complexity analysis is the first filter for deciding whether a solution is worth implementing.
- Day 0 prepares students for the technical sequence that follows in Days 1-5.

CP4 Reading Connections

- CP4 Section 1.2.2, *International Collegiate Programming Contests (ICPC)*, for the contest context.
- CP4 Section 1.3, *Tips to be Competitive*, especially Sections 1.3.2-1.3.7 on problem identification, analysis, language mastery, testing, practice, and teamwork.
- CP4 Section 1.4.1, *Anatomy of a Programming Contest Problem*, and Section 1.4.4, *Getting Our First Accepted (AC) Verdict*, for first-solve workflow.
- CP4 Sections 1.3.3, 2.2.1, 2.2.5, 2.2.6, 2.3.1, and 2.3.2 for the Day 1 preview material.
- CP4 Chapters 3-7 for the remaining weekly roadmap: search, greedy, dynamic programming, graphs, number theory, and geometry.